

# Minerva: una Pipeline di Parsing e Valutazione Automatica per Sorgenti Web Eterogenee

Laura De Michelis – Matricola 2119887 – [demichelis.2119887@studenti.uniroma1.it](mailto:demichelis.2119887@studenti.uniroma1.it)  
Andrea Giuliotti – Matricola 2107814 – [giuliotti.2107814@studenti.uniroma1.it](mailto:giuliotti.2107814@studenti.uniroma1.it)  
Alessandro Di Nitto – Matricola 2109155 – [dinitto.2109155@studenti.uniroma1.it](mailto:dinitto.2109155@studenti.uniroma1.it)

Sapienza Università di Roma

## 1 Obiettivo del Progetto

Con questo progetto abbiamo costruito una pipeline completa che scarica pagine da quattro domini web diversi (Wikipedia in italiano, AppleVis, Basketball-Reference e OndaRock), ne estrae il testo utile, lo confronta con un Gold Standard scritto da noi e lo fa valutare anche da un LLM locale. Tutto il sistema è esposto tramite API REST e una Web UI, e gira in quattro container Docker (backend, frontend, database, LLM).

## 2 Organizzazione del Codice

Il backend (FastAPI, porta 8003) sta in `backend/src/` ed è diviso in quattro parti. In `parsers/` c'è una classe astratta `BaseDomainParser` che si occupa di tutto quello che è comune a ogni sito (aprire il browser di Crawl4AI, scaricare l'HTML, riparsarlo da una stringa già salvata), mentre ogni dominio ha la sua sottoclasse con il selettore CSS giusto, le esclusioni per togliere il rumore e la pulizia finale del Markdown – abbiamo capito quali selettori usare guardando a mano l'HTML di ogni sito. Un `ParserFactory` sceglie il parser giusto in base al dominio dell'URL. In `evaluation/` c'è tutto quello che serve a ripulire il Markdown prima del confronto, calcolare le metriche e parlare con Ollama per il giudizio dell'LLM. In `db/` c'è la connessione a MariaDB, lo schema e le funzioni per leggere/scrivere, incluso lo script che popola il database all'avvio. In `models/` ci sono gli schemi Pydantic di input/output di ogni endpoint. Il frontend (FastAPI + Jinja2, porta 8004) ha quattro pagine (home, parser & evaluation, gold standard builder, stats) e parla *solo* con le API REST del backend, mai direttamente col database.

## 3 Valutazione Parser

Come richiesto abbiamo usato `token_level_eval` (precision, recall, F1 sui token separati da spazio, dopo aver tolto il Markdown da entrambi i testi), e in più abbiamo aggiunto ROUGE-1: a differenza della prima metrica, che guarda solo se una parola c'è o no, ROUGE-1 conta anche quante volte compare, quindi si accorge anche se il parser ripete o salta pezzi di testo.

Gli F1 medi che abbiamo ottenuto sulle 10 pagine di ogni dominio sono: `it.wikipedia.org` 0.90, `www.applevis.com` 0.97, `www.basketball-reference.com` 0.90, `www.ondarock.it` 0.99. Per `basketball-reference.com` abbiamo deciso di far coprire al Gold Standard solo il blocco anagrafico in alto alla pagina (`#meta`: ruolo, squadra, dati di nascita, statistiche riassuntive), e non le tabelle enormi di statistiche partita per partita che seguono, perché quelle sono dati veri ma non “testo informativo di sintesi” nel senso che chiede la consegna.

Durante la costruzione e il controllo del Gold Standard abbiamo trovato due problemi interessanti. Il primo: il parser HTML della libreria standard di Python si fermava senza dire niente a metà di una pagina con un markup un po' irregolare, tagliando via il 95% del contenuto reale – ce ne siamo accorti

confrontando la lunghezza che ci aspettavamo con quella che uscivano, e abbiamo risolto passando a un parser HTML più permissivo. Il secondo: anche quando l'estrazione è sostanzialmente giusta, piccoli residui di formattazione Markdown (uno spazio in più o in meno prima dei due punti, parole attaccate ai bordi di un grassetto o un corsivo dopo aver tolto i simboli) fanno perdere qualche punto di F1: è un limite di come si tokenizza il testo (a spazi), più che un vero difetto del parser – l'abbiamo verificato controllando a mano i token che non combaciavano tra estrazione e Gold Standard.

## 4 Valutazione dei Judge

Come LLM Judge abbiamo scelto `llama3.2:3b` (Meta, circa 2GB di RAM) tra i cinque modelli proposti, principalmente perché è il più veloce su CPU e la macchina su cui abbiamo sviluppato non è potentissima: una singola valutazione ci mette da circa 1 a circa 4 minuti, a seconda che il modello sia già caricato in RAM o debba essere ricaricato da disco. Il prompt chiede una risposta in JSON con `score` (da 1 a 5) e `feedback`, con temperatura bassa (0.1) per avere un giudizio il più possibile coerente a parità di input. Per il parsing della risposta abbiamo previsto quattro livelli di fallback in cascata (JSON diretto, poi il primo blocco `{ . . . }` che si trova nel testo, poi i singoli campi via regex, infine una cifra isolata), e solo se proprio niente funziona ricadiamo su un punteggio neutro.

Una cosa che ci ha colpito facendo i test: su tutte e 40 le pagine del Gold Standard il punteggio è quasi sempre 4/5, indipendentemente dal vero F1 (che va da 0.85 a 0.99). Per capire se fosse un bug abbiamo fatto un test di controllo dandogli un testo completamente scollegato dal Gold Standard, e lì il punteggio è sceso correttamente a 2/5, con un feedback che citava proprio l'argomento sbagliato – quindi il modello legge davvero il contenuto, non restituisce sempre lo stesso numero. Semplicemente sembra che riservi il 5/5 quasi solo a corrispondenze testuali identiche, e tratti qualunque piccola imperfezione (anche minima) allo stesso modo. È un comportamento abbastanza noto dei modelli piccoli usati come giudici automatici a bassa temperatura, non un difetto della nostra pipeline.

Per curiosità abbiamo provato anche altri modelli oltre a `llama3.2:3b`: `qwen3:4b` (non compatibile con il nostro codice così com'è, perché restituisce il ragionamento in un campo `thinking` separato invece che in `response`), `phi4-mini` (risultati peggiori, punteggi ancora meno informativi), `gemma3:e2b` (7GB, troppo pesante: il caricamento del modello saturava la RAM e faceva andare in timeout il browser usato per lo scraping) e `ministral-3:3b` (3GB, dimensione simile a `llama3.2:3b`, ma con generazione a circa 1 token al secondo sulla nostra CPU: una singola valutazione ha impiegato oltre 7 minuti e la richiesta successiva ha fatto crashare il processo interno di Ollama). Questi test confermano che il collo di bottiglia è l'hardware della macchina di sviluppo (CPU senza GPU dedicata, RAM condivisa tra Docker, Ollama e il browser per lo scraping) più che la scelta del modello, e che `llama3.2:3b` resta il compromesso più stabile tra i cinque proposti.

Abbiamo anche provato a migliorare il prompt del judge dando una rubrica esplicita (criteri diversi per ogni punteggio da 1 a 5, invece della richiesta generica) su tre pagine reali con F1 molto diverso tra loro (1.00, 0.92, 0.81): il punteggio restituito è stato 4/5 in tutti e tre i casi, sia col prompt vecchio che con quello nuovo. Il feedback testuale cambiava correttamente da un caso all'altro (segno che il modello legge davvero i testi), ma il punteggio numerico no. Consideriamo quindi questo comportamento un limite di calibrazione del modello più che un problema risolvibile lato nostro senza introdurre un aggiustamento manuale del punteggio, cosa che abbiamo deciso di non fare per non falsare il giudizio dell'LLM.

## 5 Organizzazione del DB

Nel database MariaDB abbiamo le due tabelle obbligatorie `web_resources` e `gold_standard`, collegate uno-a-uno con una chiave esterna su `url` (con cascade in eliminazione), più due tabelle nostre: `evaluations` (i risultati di `token_level_eval` e ROUGE-1 per ogni pagina) e `llm_judgments` (punteggio, feedback e nome del modello usato per il giudizio). Tutte le query passano da un livello di repository e sono parametrizzate. All'avvio del backend, uno script di seed legge tutti i file `gs_data/*.json` e riempie `web_resources` e `gold_standard` con un upsert, così l'HTML

grezzo di ogni pagina resta sempre disponibile per rifare il parsing e confrontarlo col `gold_text` quando vogliamo, senza dover ricaricare niente.